

MQTT 패킷을 읽는 법

메시지는 어떤 바이트로 전달될까?
제어 헤더부터 연결·구독·QoS까지

01	02	03	04
구조	인코딩	흐름	적용
Header·Payload	비트·길이·문자열	연결·발행·확인	Explorer·Unity

기억할 내용

영문 공학 용어를 바이트 예제와 연결해 이해합니다.

원문의 주제를 한국어 학습 순서로 연결합니다

원문에서 다루는 주제	이 자료에서 확인할 것
Message format / Header	패킷의 세 영역과 첫 바이트
Control field / Flags	종류 코드와 DUP·QoS·RETAIN
Remaining Length	128진 가변 길이 계산
CONNECT / DISCONNECT example	직접 구성한 연결 패킷 해독
Packet summary / Wireshark	패킷별 필드와 전송 계층 구분

기억할 내용

주제 흐름은 원문을 참고하고, 도식·계산·Unity 예제는 새로 구성했습니다.

근거 · [참고 원문](#) · 세부 규칙은 [OASIS MQTT 3.1.1 · 2.3장](#)로 확인

JSON과 MQTT 패킷은 같은 것이 아닙니다

응용 데이터

```
{ "rpm": 1420 }
```

우리가 보내려는 값입니다. JSON은 응용 프로그램이 선택한 표현 방식입니다.

전송되는 패킷

종류·길이	토픽 등	값
Fixed Header	Variable Header	Payload
필수	유형에 따라	유형에 따라

라이브러리가 제어용 바이트를 붙여 보냅니다.

기억할 내용

Explorer에서 입력하는 JSON은 PUBLISH의 Payload이며, CONNECT·PINGREQ도 MQTT 패킷입니다.

고정 헤더는 최소 2바이트입니다

첫 바이트	이후 바이트	그 뒤
Type + Flags	Remaining Length	Variable Header + Payload
1 byte	1~4 bytes	Remaining Length만큼

$$\text{MQTT 전체 길이} = 1 + \text{길이 필드의 바이트 수} + \text{Remaining Length}$$

기억할 내용

Fixed Header라는 이름이 길이가 항상 2바이트라는 뜻은 아닙니다. 실제 범위는 2~5바이트입니다.

근거 · [OASIS MQTT 3.1.1 · 2.1-2.2.3](#)

첫 바이트를 두 부분으로 나눕니다

bit 7~4 Packet Type 상위 4비트	bit 3~0 Flags 하위 4비트
---	---

CONNECT: 0001 0000 = 0x10 PUBLISH: 0011 0000 = 0x30 (QoS 0, DUP=0, RETAIN=0) SUBSCRIBE: 1000 0010 = 0x82

기억할 내용

패킷 종류 8과 실제 첫 바이트 0x82를 구분하세요. 하위 플래그의 규칙도 함께 적용합니다.

근거 · 표준의 첫 바이트 규칙을 적용한 독자 계산 예제

주요 패킷 코드 · 연결과 데이터

종류 번호	패킷	첫 바이트 · 16진수
1	CONNECT · 연결 요청	10
2	CONNACK · 연결 응답	20
3	PUBLISH · 데이터 전달	30~3D 중 허용 조합
4	PUBACK · QoS 1 확인	40
5	PUBREC · QoS 2 수신 확인	50
6	PUBREL · QoS 2 다음 단계	62
7	PUBCOMP · QoS 2 완료	70

기억할 내용

PUBLISH는 플래그에 따라 첫 바이트가 달라집니다. QoS 비트 11과 QoS 0의 DUP=1은 허용되지 않습니다.

근거 · [OASIS MQTT 3.1.1 · 2.2.1-2.2.2-3.3.1](#)

주요 패킷 코드 · 구독과 연결 유지

종류 번호	패킷	첫 바이트 · 16진수
8	SUBSCRIBE · 구독 요청	82
9	SUBACK · 구독 결과	90
10	UNSUBSCRIBE · 구독 해제	A2
11	UNSUBACK · 해제 확인	B0
12	PINGREQ · 연결 확인 요청	C0
13	PINGRESP · 연결 확인 응답	D0
14	DISCONNECT · 정상 종료 통지	E0

기억할 내용

이 자료는 MQTT 3.1.1 기준입니다. MQTT 5.0의 AUTH·Properties·Reason Code 체계와 구분합니다.

근거 · [OASIS MQTT 3.1.1 · 2.2.1-2.2.2](#)

PUBLISH 플래그 · DUP·QoS·RETAIN

bit 3 DUP 재전송 표시	bit 2~1 QoS 00 / 01 / 10	bit 0 RETAIN 보존 관련 표시
-------------------------------	---------------------------------------	------------------------------------

첫 바이트	읽는 방법
0x30	QoS 0 · RETAIN=0
0x32	QoS 1 · RETAIN=0
0x35	QoS 2 · RETAIN=1

기억할 내용

DUP는 응용 명령 ID가 아닙니다. 재발행한 과거 ON 명령의 실행 여부는 응용 코드에서 판단합니다.

Remaining Length를 계산합니다

$321 = 65 + 2 \times 128$

첫 바이트: $65 + 128 = 193 = 0xC1$

둘째 바이트: $2 = 0x02$

결과: C1 02

각 바이트의 하위 7비트가 값이고, 최상위 비트는 다음 바이트가 더 있는지 나타냅니다.

기억할 내용

C1 01을 같은 방식으로 해독하면 193입니다. 321의 올바른 인코딩은 C1 02입니다.

근거 · 독자 계산으로 검증 · [OASIS MQTT 3.1.1 · 2.2.3](#)의 예제와 대조

길이가 바뀌면 헤더도 커집니다

Remaining Length · 10진수	인코딩 · 16진수	MQTT 전체 길이
0	00	2 bytes
127	7F	129 bytes
128	80 01	131 bytes
321	C1 02	324 bytes

Remaining Length

321

C1 02 · 전체 324 bytes

기억할 내용

길이 필드 자체와 첫 제어 바이트는 Remaining Length에 포함하지 않습니다.

문자 수와 UTF-8 바이트 수

"unity-b"

길이: 7 bytes

00 07

75 6E 69 74 79 2D 62

앞의 두 바이트는 문자열 길이

- 00 07은 뒤따르는 UTF-8 데이터의 길이입니다.
- 길이 접두어 2바이트는 그 값에 포함하지 않습니다.
- 한글은 글자 수와 UTF-8 바이트 수가 다릅니다.

"가" → EA B0 80 → 3 bytes

기억할 내용

Remaining Length의 가변 길이 인코딩과 문자열의 2바이트 길이 접두어는 서로 다른 규칙입니다.

근거 · 독자 문자열 인코딩 예제 · MqttLiteClient.BuildMqttString과 대조

연결 패킷에 들어가는 정보

Variable Header

필드	이번 예제
Protocol Name	MQTT
Protocol Level	04 · MQTT 3.1.1
Connect Flags	02 · Clean Session=1
Keep Alive	00 1E · 30초

Payload

- Client ID: unity-b
- Will Topic·Will Message는 설정 시 포함
- User Name·Password는 설정 시 포함

이번 계산은 Will과 인증 정보를 사용하지 않습니다.

기억할 내용

CONNECT에는 Client ID를 담는 Payload가 있습니다. CONNACK에는 MQTT 3.1.1 Payload가 없습니다.

근거 · [HiveMQ · 연결 설명](#)

Connect Flags는 별도의 바이트입니다

비트	이름	이번 예제
7 / 6	User Name / Password Flag	0 / 0
5	Will Retain	0
4~3	Will QoS	00
2	Will Flag	0
1	Clean Session	1
0	Reserved	0

0000 0010 = 0x02

기억할 내용

CONNECT의 첫 바이트 0x10과 가변 헤더 안의 Connect Flags 0x02는 위치와 역할이 다릅니다.

21바이트 연결 패킷 해독

10 13
00 04 4D 51 54 54 04 02 00 1E
00 07 75 6E 69 74 79 2D 62

구간	내용	길이
10 13	CONNECT / Remaining Length=19	2 bytes
00 04 ... 00 1E	프로토콜·버전·플래그·Keep Alive	10 bytes
00 07 ... 2D 62	Client ID = unity-b	9 bytes

기억할 내용

10 + 9 = 19 = 0x13. 전체는 2 + 19 = 21바이트입니다.

근거 · 직접 구성한 MQTT 3.1.1 CONNECT 예제 · 캡처 데이터 아님

CONNACK와 종료 패킷

연결 수락 예

20 02 00 00

20 : CONNACK
02 : Remaining Length
00 : Session Present=0
00 : 연결 수락

정상 종료

E0 00

E0 : DISCONNECT
00 : 뒤 따르는 데이터 없음

MQTT 3.1.1에서 별도의 DISCONNECT 확인 패킷은 없습니다.

기억할 내용

모든 MQTT 패킷에 확인 응답이 있는 것은 아닙니다. QoS 0 PUBLISH에도 MQTT 확인 응답은 없습니다.

근거 · [HiveMQ · 연결 설명](#) · [OASIS MQTT 3.1.1 · 3.14](#)

PUBLISH를 직접 읽어봅니다

Header	Topic	Payload
30 0D	00 07 + lab/rpm	1420
QoS 0 / RL=13	9 bytes	4 bytes

30 0D 00 07 6C 61 62 2F 72 70 6D 31 34 32 30

기억할 내용

Payload의 "1420"은 네 글자의 UTF-8 데이터입니다. 숫자 1420을 2바이트 정수로 넣은 것이 아닙니다.

근거 · 독자 예제: lab/rpm 토픽 · Payload 1420 · QoS 0

QoS 1이면 Packet Identifier가 추가됩니다

32 0F 00 07 6C 61 62 2F 72 70 6D 00 01 31 34 32 30	
달라진 부분	이유
30 → 32	발행 QoS 0 → 1
0D → 0F	Packet Identifier 2바이트 추가
00 01	이 교환의 Packet Identifier=1

기억할 내용

식별자는 QoS 1:2 PUBLISH에 있습니다. 수신 파서도 달라져야 하므로 헤더 비트만 바꾸면 안 됩니다.

QoS 0·1·2의 교환 비용

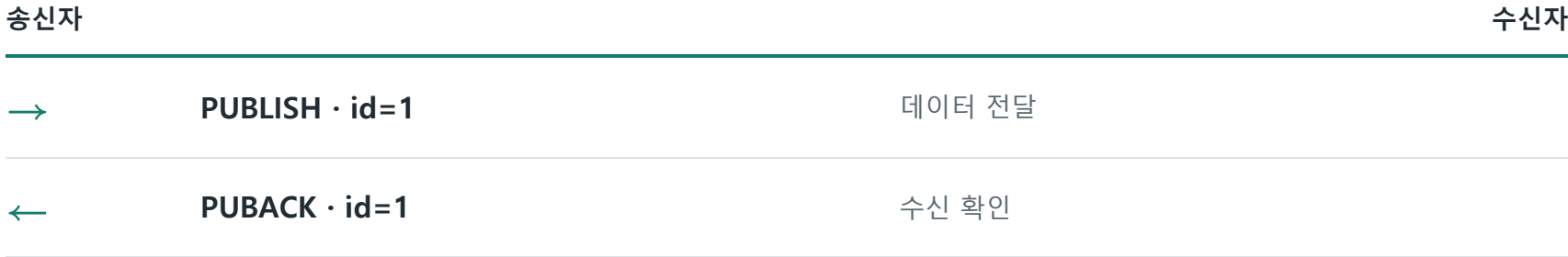
수준	전달 의미	교환 패킷
0	최대 한 번 · 유실 가능	PUBLISH
1	최소 한 번 · 중복 가능	PUBLISH / PUBACK
2	프로토콜 범위의 중복 전달 방지	PUBLISH / PUBREC / PUBREL / PUBCOMP

기억할 내용

QoS는 JSON의 필드가 아니라 MQTT 전송 설정입니다. 확인 절차에는 통신·상태 관리 비용이 듭니다.

근거 · [EMQX · QoS 교환](#)

QoS 1의 확인 응답



```
PUBACK 예 제 : 40 02 00 01
```

기억할 내용

PUBACK는 해당 MQTT 전달의 확인입니다. Unity의 화면 적용이나 모터의 물리 동작 완료를 뜻하지 않습니다.

근거 · [EMQX · QoS 교환](#) · 독자 Packet Identifier 예제

QoS 2의 네 단계

송신자		수신자
→	PUBLISH · id=1	데이터 전달
←	PUBREC · id=1	수신 확인
→	PUBREL · id=1	후속 처리 단계
←	PUBCOMP · id=1	교환 완료

기억할 내용

재연결 시 미완료 교환과 중복 패킷을 처리할 상태가 필요합니다. QoS 2도 응용 명령의 멍등성을 대신하지 않습니다.

근거 · [EMQX · QoS 교환](#)

SUBSCRIBE와 SUBACK를 구분합니다

QoS 1 구독 요청

```
82 0C 00 01
00 07 6C 61 62 2F 72 70 6D 01
```

Packet Identifier=1, 필터 lab/rpm, 요청 QoS=1.

구독 승인 예

```
90 03 00 01 01
```

같은 식별자에 대해 QoS 1이 허용됐다는 응답입니다. 실패 코드는 0x80입니다.

기억할 내용

SUBSCRIBE의 고정 플래그 0010과 Payload의 요청 QoS는 서로 다른 필드입니다.

근거 · [HiveMQ](#) · 발행·구독 · 독자 구독 패킷 예제

두 전달 구간의 QoS는 독립적입니다

01

Explorer

QoS 2 발행

02

Broker

구독 조건 적용

03

Unity B

QoS 0 구독이면 0으로 수신

- 발행 측의 QoS 설정이 B의 구독 설정을 바꾸지는 않습니다.
- Broker의 구독 승인 결과도 확인해야 합니다.
- 각 구간의 Packet Identifier는 응용 요청 ID와 별개입니다.

기억할 내용

Explorer에서 QoS 2를 선택한 것만으로 종단 간 QoS 2 실습이 완성되지는 않습니다.

근거 · [HiveMQ](#) · 발행·구독 · 우리 테스트 구조에 적용한 설명

연결 유지와 구독 해제

동작	패킷 예 · 16진수	의미
PINGREQ	C0 00	연결 확인 요청
PINGRESP	D0 00	연결 확인 응답
UNSUBSCRIBE	A2 + 길이 + 식별자 + 필터	해당 구독 해제 요청
UNSUBACK	B0 02 00 01	식별자 1의 해제 확인

기억할 내용

구독 해제와 연결 종료는 다릅니다. Keep Alive는 연결 생존을 보며 센서 데이터 최신성을 직접 검사하지 않습니다.

근거 · [OASIS MQTT 3.1.1 · 3.10~3.14](#)

TCP ACK와 MQTT ACK는 다른 계층입니다

TCP ACK

바이트 스트림의 수신을 확인합니다. 한 TCP 세그먼트와 한 MQTT 패킷의 경계가 항상 같지는 않습니다.

MQTT ACK

CONNACK·SUBACK·PUBACK처럼 특정 MQTT 절차의 결과를 나타냅니다.

MQTT 길이 ≠ 캡처된 전체 프레임 길이

기억할 내용

네트워크·링크 헤더와 캡처 환경에 따라 부가 길이가 달라집니다. 오버헤드를 항상 56바이트로 고정하지 않습니다.

근거 · 원문의 Wireshark 주제를 바탕으로 계층 차이를 재구성 · [Wireshark · MQTT 필드](#)

Explorer와 Wireshark에서 볼 것

도구	관찰 대상	학습 질문
MQTT Explorer	토픽·Payload·발행 QoS·Retain	내가 어떤 값을 보냈는가?
Wireshark	CONNECT·PUBLISH·SUBACK 등 패킷	어떤 절차와 바이트로 전달됐는가?
Unity Console	수신·JSON 해석·적용 결과	화면까지 처리됐는가?

Wireshark 표시 필터: mqtt

PUBLISH만: mqtt.msgtype == 3

기억할 내용

교육용 평문 연결에서 캡처합니다. 로컬 Broker는 루프백 경로를, TLS는 복호화 준비 여부를 고려해야 합니다.

근거 · [Wireshark · MQTT 필터](#) · 교육 환경 적용 설명

우리 코드에서 오늘 배운 바이트 찾기

코드 위치	대응 내용
ConnectAsync	0x10 · MQTT 문자열 · 버전 04 · Flags 02
PublishAsync	0x30 또는 0x31 · 현재 QoS 0
SubscribeAsync	0x82 · 요청 QoS 00
EncodeRemainingLength	7비트씩 나누는 길이 인코딩
BuildMqttString	UTF-8 바이트와 2바이트 길이 접두어
KeepAliveLoopAsync	0xC0 · PINGREQ

기억할 내용

현재 최소 클라이언트에는 QoS 1·2 확인 교환과 SUBACK 결과 검증이 없습니다.

근거 · 코드 대조: 사용 중인 프로젝트의 MqttLiteClient.cs · 2026-09-13 확인

원문을 읽을 때 함께 기억할 보완점

주제	정확하게 구분할 내용
Fixed Header	항상 2바이트가 아니라 최소 2, 최대 5바이트
길이 321	C1 02 · C1 01은 193
문자열	Client ID·Topic·User Name은 UTF-8, Password는 3.1.1에서 바이너리 필드
확인 응답	QoS 0 PUBLISH·DISCONNECT는 MQTT ACK 없음
Payload	CONNECT에는 있고 CONNACK에는 없음
ACK 길이	PINGRESP는 2바이트, CONNACK·PUBACK는 4바이트

기억할 내용

패킷 이름·버전·필드 위치를 함께 확인하면 모호한 설명을 스스로 검산할 수 있습니다.

확인 문제와 해설

문제	해설
0x32는 어떤 패킷인가?	PUBLISH, QoS 1, DUP=0, RETAIN=0
Remaining Length=128이면 전체 길이는?	1 + 2 + 128 = 131바이트
E0 00 뒤에 Payload가 있는가?	없음 · 3.1.1 정상 종료 통지
CONNECT의 04는 문자 "4"인가?	아님 · 프로토콜 수준을 나타내는 수치 4
JSON에 "qos":2를 쓰면 QoS 2인가?	아님 · 응용 데이터만 바뀜

기억할 내용

바이트 구조를 이해하면 설정·전송·수신·응용 실행을 구분할 수 있습니다.

근거 · 교육용 독자 예제 · 실제 패킷 캡처가 아닌 계산·구성 예시

참고 자료

- [Steve's Internet Guide · Understanding the MQTT Protocol Packet Structure](#)
- [OASIS MQTT 3.1.1 · 공식 규격](#)
- [HiveMQ · 연결 설명](#)
- [HiveMQ · 발행·구독](#)
- [EMQX · QoS 교환](#)
- [Wireshark · MQTT 필드](#)

원문 주제를 참고한 한국어 해설입니다. 그림은 새로 구성했으며 바이트 예제는 직접 계산했습니다. 원문 본문은 HTTP 경로로 확인했습니다.

기억할 내용

이 자료의 바이트 예제는 MQTT 3.1.1 기준이며, HTML과 PDF는 인터넷 없이 열 수 있습니다.

근거 · 확인일 2026-09-13 · 출처 링크 열기는 인터넷 연결 필요